

系统开发工具基础第二周实验报告

姓名： 刘静雅

学号： 24160001055

日期： 2026 年 8 月 31 日

仓库地址：

<https://github.com/Llj159-lab/sys-dev-tools-lab>

目录

1	实验环境与目录结构	2
2	q05: 后台任务、信号处理与清理	2
3	q06: 编辑器辅助功能与静态检查	4
4	q07: 调试错误并使用测试验证	7
5	q08: 性能分析与代码优化	8
6	本周实验小结	11

1 实验环境与目录结构

本周实验在 WSL 的 Bash 环境中完成, 仓库目录为 `~/sys-dev-tools-lab/week02`。
本周实验目录结构如下:

```
week02/  
|-- q05/  
|-- q06/  
|-- q07/  
|-- q08/  
|-- report_assets/  
`-- report.tex
```

实验过程中分别使用了 Shell 后台任务与信号处理、Python 测试与编辑器辅助功能、调试和单元测试, 以及性能分析和代码优化等方法。

2 q05: 后台任务、信号处理与清理

原题

编写并运行后台任务程序, 观察后台任务的运行状态; 使用进程管理命令获取进程信息, 并通过信号终止任务; 编写清理脚本, 删除或整理实验过程中产生的临时文件, 同时保存标准输出和标准错误输出。

实验目的

本题主要练习 Shell 中后台任务的启动与管理、进程号的获取、信号的发送, 以及清理脚本的编写。实验重点是理解后台任务与当前终端之间的关系, 并掌握使用脚本自动完成清理工作的基本方法。

实验步骤

首先进入 q05 目录:

```
cd ~/sys-dev-tools-lab/week02/q05
```

然后运行后台任务, 并观察任务状态:

```
bash worker.sh > stdout.log 2> stderr.log &  
jobs -l  
jobs -p
```

其中，符号 & 使任务在后台执行；jobs -l 可以查看当前 Shell 管理的后台任务及其进程号；jobs -p 只输出后台任务的进程号，便于后续操作。

实验中通过进程号向任务发送终止信号：

```
pid=$(jobs -p | head -n 1)
kill -TERM "$pid"
```

TERM 表示请求进程正常终止。与强制终止相比，这种方式更适合测试程序是否能够执行清理逻辑。

最后运行清理脚本，并查看清理日志：

```
bash cleanup.sh
cat cleanup.log
```

实验结果

实验成功启动了后台任务，并使用 jobs -l 和 jobs -p 获取了任务信息。随后使用 kill -TERM 向后台进程发送终止信号，任务停止运行。清理脚本执行后生成了 cleanup.log，标准输出和标准错误分别保存到 stdout.log 与 stderr.log。

```
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q05$ cd ~/sys-dev-tools-lab/week02/q05
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q05$ chmod +x worker.sh
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q05$ ./worker.sh > stdout.log 2> stderr.log
^Z
[1]+  Stopped                  ./worker.sh > stdout.log 2> stderr.log
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q05$ bg
[1]+  ./worker.sh > stdout.log 2> stderr.log &
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q05$ jobs
[1]+  Running                  ./worker.sh > stdout.log 2> stderr.log &
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q05$
```

图 1: 启动后台任务并查看任务状态

```
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q05$ jobs -p
8570
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q05$ kill -TERM $(jobs -p)
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q05$ wait
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q05$ cat cleanup.log
CLEAN_EXIT
CLEAN_EXIT
CLEAN_EXIT
CLEAN_EXIT
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q05$ cat stdout.log
cat: stdout.log: No such file or directory
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q05$ cat stderr.log
```

图 2: 终止后台任务并执行清理脚本

分析

本题说明了后台任务管理与普通前台命令的差异。后台任务启动后不会阻塞当前终端，但仍然属于当前 Shell 的任务管理范围。通过 jobs 可以查询任务，通过进程号可

以进一步使用 `kill` 发送信号。将标准输出和标准错误分别保存到文件，有助于区分正常运行信息和异常信息。清理脚本则将重复性的文件处理过程自动化，减少了手工操作产生错误的可能性。

3 q06: 编辑器辅助功能与静态检查

原题

使用编辑器阅读和修改 Python 项目，完成符号查找、跳转到定义、查找引用和符号重命名等操作；运行静态检查工具，发现并处理未使用的导入或其他代码问题；使用测试命令确认修改没有破坏原有功能。

实验目的

本题练习现代编辑器在软件开发中的辅助作用。实验内容包括代码导航、符号重命名、引用查找和静态检查。通过这些工具，可以提高阅读代码、定位问题和修改代码的效率。

实验步骤

首先进入 q06 目录：

```
cd ~/sys-dev-tools-lab/week02/q06
```

项目包含以下文件：

- `math_utils.py`: 数学计算函数；
- `test_math_utils.py`: 对应的测试代码；
- `app.py`: 调用相关函数的应用代码。

打开项目后，首先在编辑器中使用“查找引用”功能，查看函数或变量在项目中的使用位置。随后使用“跳转到定义”功能，从调用位置跳转到函数的定义位置。

完成符号重命名操作后，需要检查所有引用是否同步更新。该步骤不能只修改一个文件中的文字，因为函数名可能同时出现在实现代码、测试代码和应用代码中。

然后运行静态检查：

```
ruff check .
```

根据检查结果删除未使用的导入，或修正其他静态问题。最后执行测试：

```
pytest -q
```

实验结果

实验完成了查找引用、跳转定义和符号重命名操作。静态检查能够发现未使用的导入问题，修改后重新执行检查，相关问题得到处理。最后运行 pytest，测试结果通过。

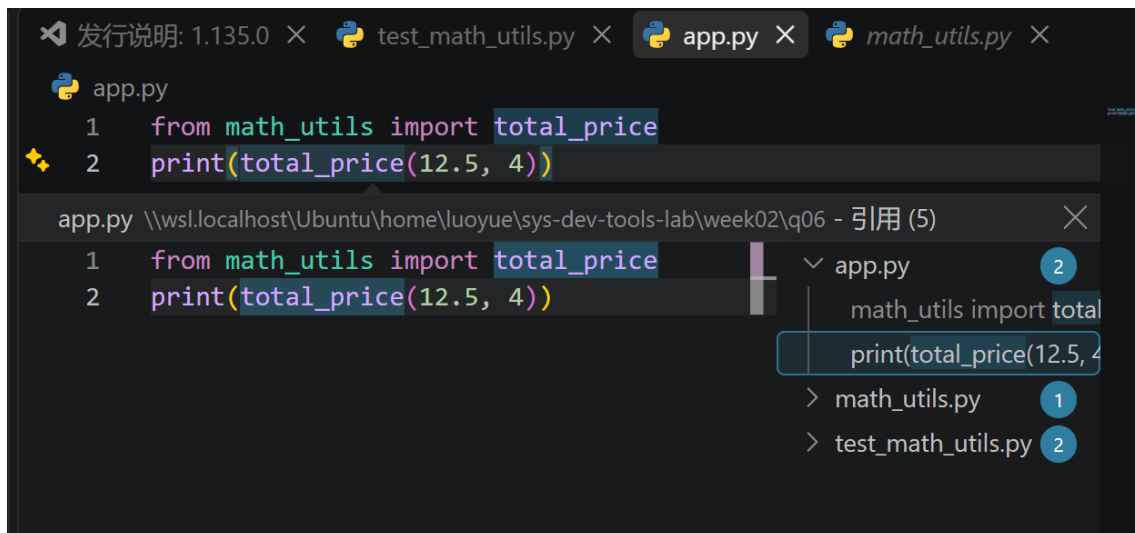


图 3: 查找符号引用

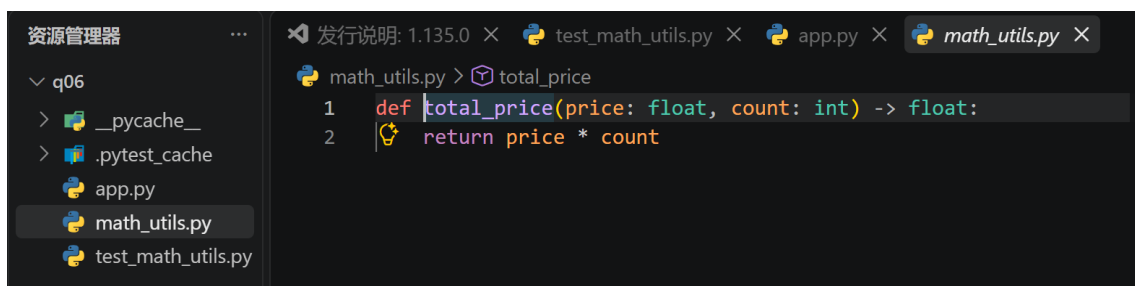


图 4: 跳转到符号定义

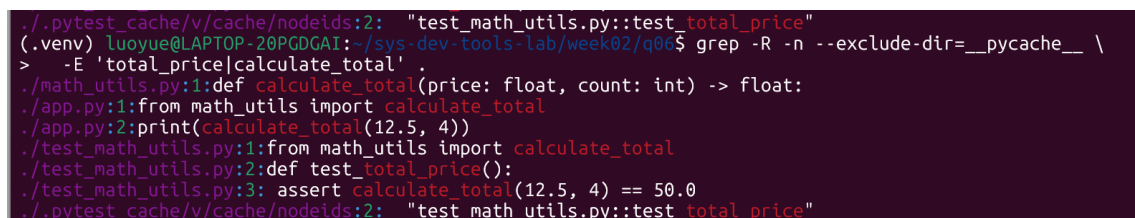


图 5: 重命名符号并同步修改引用

```
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q06$ ruff check .
I001 [*] Import block is un-sorted or un-formatted
--> app.py:1:1
1 | / import os
2 | | from math_utils import calculate_total
3 | |
4 | | print(calculate_total(12.5, 4))
help: Organize imports
1 | import os
2 | +
3 | from math_utils import calculate_total

F401 [*] `os` imported but unused
--> app.py:1:8
1 | import os
2 |   ^^^
3 | from math_utils import calculate_total
help: Remove unused import: `os`
- import os
1 | from math_utils import calculate_total

I001 [*] Import block is un-sorted or un-formatted
--> test_math_utils.py:1:1
1 | from math_utils import calculate_total
2 | def test_total_price():
3 |     assert calculate_total(12.5, 4) == 50.0
help: Organize imports
1 | from math_utils import calculate_total
2 | +
3 | +
4 | def test_total_price():

Found 3 errors.
[*] 3 fixable with the `--fix` option.
```

图 6: 静态检查发现未使用的导入

```
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q06$ ruff format .
3 files reformatted
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q06$ ruff check .
All checks passed!
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q06$ pytest -q
. [100%]
1 passed in 0.01s
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q06$ python app.py
50.0
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q06$
```

图 7: 修改后重新执行检查和测试

分析

编辑器的代码导航功能可以将源代码中的定义、调用关系和引用位置联系起来。查找引用适合分析修改范围，跳转定义适合快速理解函数实现，符号重命名适合对项目中的标识符进行一致性修改。静态检查可以在运行测试前发现一部分代码质量问题，而 `pytest` 则从功能角度验证程序行为。将编辑器功能、静态检查和自动化测试结合起来，可以形成较完整的修改验证流程。

4 q07: 调试错误并使用测试验证

原题

运行给定的排序程序和测试代码，观察测试失败信息；使用调试工具定位错误原因，修改程序中的缺陷；重新运行测试，确认修复后的程序满足测试要求。

实验目的

本题主要练习根据测试失败信息定位程序错误，并通过调试和单元测试验证修复结果。实验重点是理解测试用例、错误堆栈和程序实际执行过程之间的关系。

实验步骤

进入 q07 目录：

```
cd ~/sys-dev-tools-lab/week02/q07
```

首先直接运行测试：

```
pytest -q
```

根据测试输出定位失败的测试用例，再打开 `merge_sort.py` 检查归并过程中左右两个子数组的下标访问。

调试时重点观察以下内容：

- 左右两个数组的起始下标；
- 循环条件是否允许访问数组末尾元素；
- 比较左右元素时使用的下标是否对应当前数组；
- 剩余元素复制过程是否完整。

修改程序后重新运行测试：

```
pytest -q
```

如果所有测试通过，再检查文件内容和最终输出，确认修复没有引入新的问题。

实验结果

初次运行测试时，排序程序出现测试失败。通过查看失败信息和调试程序执行过程，定位到归并排序实现中的下标访问错误。修改相关代码后，重新运行 `pytest`，测试全部通过。

```
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q07$ nano merge_sort.py
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q07$ python merge_sort.py
[1, 5, 4, 3, 4, 5, 6, 9]
```

图 8: 修复前运行测试的失败结果

```
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q07$ nano merge_sort.py
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q07$ python merge_sort.py
[1, 1, 2, 3, 4, 5, 6, 9]
```

图 9: 修改排序程序后的代码状态

```
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q07$ nano test_merge_sort.py
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q07$ pytest -q
..
2 passed in 0.01s
```

图 10: 修复后测试全部通过

分析

调试的关键不是直接修改表面现象，而是根据错误信息确定错误发生的位置，再分析程序状态和控制流程。对于归并排序，需要同时关注左右子数组的边界和元素比较过程。测试用例提供了可重复的验证条件，因此修复代码后可以立即通过同一组测试判断结果。该过程体现了“测试失败、定位原因、修改代码、重新测试”的基本调试闭环。

5 q08: 性能分析与代码优化

原题

生成测试数据，分别运行优化前后的单词频率统计程序；使用 `time` 测量运行时间，并使用 Python 性能分析工具定位程序中的主要耗时部分；在保持输出结果一致的前提下完成优化，并比较优化前后的运行结果。

实验目的

本题练习性能测试和性能分析方法。实验要求先建立可重复的输入数据，再测量优化前的程序性能，使用分析工具定位主要耗时位置，完成优化后再次测量，并验证优化前后输出一致。

实验步骤

进入 q08 目录并生成测试数据：

```
cd ~/sys-dev-tools-lab/week02/q08
python generate_words.py
wc -w words.txt
```

生成固定规模的输入文件后，先运行优化前的程序两次：

```
time python wordfreq_slow.py words.txt > before1.out 2> before1.time
time python wordfreq_slow.py words.txt > before2.out 2> before2.time
```

使用两次运行结果可以减少单次运行波动对判断的影响。随后使用 cProfile 分析优化前程序：

```
python -m cProfile -s cumulative wordfreq_slow.py words.txt \
    > profile_before.txt
```

分析报告中的累计时间和调用次数，找出耗时较多的函数。优化时重点检查是否存在重复的线性查找、重复遍历数据或不必要的函数调用。

优化后的程序使用更适合频率统计的数据结构，避免对已经出现的单词重复进行线性搜索，并且没有将输入规模写死在程序逻辑中。

优化后再次运行两次：

```
time python wordfreq.py words.txt > after1.out 2> after1.time
time python wordfreq.py words.txt > after2.out 2> after2.time
```

最后比较输出：

```
diff -u before1.out after1.out
diff -u before2.out after2.out
```

如果没有输出差异，说明优化前后的结果一致。对两次运行时间进行比较时，可将两次数据排序后取中位数，避免只依据某一次运行结果判断性能。

实验结果

实验生成了 words.txt，并保存了优化前后的运行输出和时间记录。通过 cProfile 获得了优化前的性能分析结果，并根据主要耗时位置修改了程序。优化后重新运行程序，

输出结果与优化前保持一致。具体运行时间和中位数应以以下文件中的真实数据为准：

- before1.time、before2.time;
- after1.time、after2.time;
- before1.out、before2.out;
- after1.out、after2.out。

```
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ nano generate_words.py
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ python generate_words.py
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ ls -lh words.txt
-rw-r--r-- 1 luoyue luoyue 144K Aug 31 15:20 words.txt
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ wc -w words.txt
30000 words.txt
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ sha256sum words.txt
9e47c07f20ee78ae80ad031c4cd1013ff644ea9517f5d16d5002ec637fbfba14 words.txt
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$
```

图 11: 生成测试单词数据

```
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ nano wordfreq.py
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ cp wordfreq.py wordfreq_slow.py
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ /usr/bin/time -f '%e' -o before1.time \
python wordfreq_slow.py > before1.out

/usr/bin/time -f '%e' -o before2.time \
python wordfreq_slow.py > before2.out
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ cat before1.out
count= 1000
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ cat before2.out
count= 1000
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ cat before1.time
0.21
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ cat before2.time
0.21
```

图 12: 优化前的运行时间记录

```
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ python -m cProfile -s cumulative wordfreq_slow.py
| tee profile_before.txt
count= 1000
1012 function calls in 0.185 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
1      0.000    0.000    0.185    0.185 {built-in method builtins.exec}
1      0.183    0.183    0.185    0.185 wordfreq_slow.py:1(<module>)
1      0.002    0.002    0.002    0.002 {method 'split' of 'str' objects}
1      0.000    0.000    0.000    0.000 {method 'read' of '_io.TextIOWrapper' objects}
1000    0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
1      0.000    0.000    0.000    0.000 <frozen codecs>:319(decode)
1      0.000    0.000    0.000    0.000 {built-in method _codecs.utf_8_decode}
1      0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
1      0.000    0.000    0.000    0.000 {built-in method _io.open}
1      0.000    0.000    0.000    0.000 {built-in method builtins.print}
1      0.000    0.000    0.000    0.000 <frozen codecs>:309(__init__)
1      0.000    0.000    0.000    0.000 {built-in method builtins.len}
1      0.000    0.000    0.000    0.000 <frozen codecs>:260(__init__)

(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ grep -nE 'ncalls|wordfreq_slow|cumtime|tottime' p
rofile_before.txt
6:  ncalls  tottime  percall  cumtime  percall filename:lineno(function)
8:  1      0.183    0.183    0.185    0.185 wordfreq_slow.py:1(<module>)
```

图 13: 优化前的 cProfile 分析结果

```
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ sha256sum words.txt
9e47c07f20ee78ae80ad031c4cd1013ff644ea9517f5d16d5002ec637fbfba14 words.txt
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ /usr/bin/time -f '%e' -o after1.time \
> python wordfreq.py > after1.out
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ /usr/bin/time -f '%e' -o after2.time \
> python wordfreq.py > after2.out
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ cat after1.out
count= 1000
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ cat after2.out
count= 1000
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ cat after1.time
0.02
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ cat after2.time
0.02
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ diff -u before1.out after1.out
```

图 14: 优化后的运行时间记录

```
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ before_median=$(awk '{sum += $1} END {printf "%.6f\n", sum/NR}' before1.time before2.time)
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ echo "$before_median"
0.210000
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ after_median=$(awk '{sum += $1} END {printf "%.6f\n", sum/NR}' after1.time after2.time)
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ echo "$after_median"
0.020000
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ awk -v before="$before_median" -v after="$after_median" 'BEGIN { printf "%.2fx\n", before/after }'
10.50x
```

图 15: 优化前后结果和时间的比较

分析

性能优化应建立在测量和分析的基础上。首先固定输入数据，然后测量程序运行时间，再使用性能分析工具定位主要耗时位置，最后针对具体瓶颈进行修改。优化后不仅要关注运行速度，还要通过输出比较确认程序功能没有改变。q08 的流程体现了“建立基准、定位瓶颈、局部优化、结果验证”的基本性能工程方法。

6 本周实验小结

本周实验综合使用了 Shell 和 Python 开发工具。q05 练习了后台任务、进程信号和清理脚本；q06 练习了编辑器中的代码导航、符号重命名、静态检查和测试；q07 通过测试失败信息完成了程序调试；q08 则完成了性能测量、性能分析和代码优化。

本周实验形成了较完整的开发工作流程：先运行程序获得现象，再通过日志、测试或分析工具定位问题，完成修改后重新验证结果。实验结果不只依赖程序能够运行，还需要通过日志、测试输出、性能记录和截图保存可复核的证据。